

ЛАБОРАТОРНА РОБОТА № 4

Хід роботи:

3.1. MongoDB Atlas та MongoDB Compass

Зареєструйтесь у хмарному сервісі MongoDB та підключіться до нього через GUI-клієнт.

Вимоги:

- Створіть акаунт на cloud.mongodb.com та безкоштовний кластер МО.
- У Network Access додайте IP 0.0.0.0/0 (дозволити з будь-якого місця роботи).
- Створіть Database User з паролем, збережіть рядок підключення формату mongodb+srv://....
- Встановіть MongoDB Compass та підключіться до кластера за отриманим рядком.
- Через Compass вручну створіть базу даних і колекцію, що відповідає вашій сутності з лаб. 3.
- Додайте кілька документів вручну через Compass переконайтесь, що вони відображаються.
- Створіть файл env у корені проєкту:
- Додайте env до .gitignore та створіть .env.example з шаблоном (без реальних паролів).

// .env.example

MONGODB_URI=mongodb+srv://username:password@cluster0.xxxxx.mongodb.net/dbname
PORT=3000

					ДУ «Житомирська політехніка».25.121.13.000 – Лр1			
Змн.	Арк.	№ докум.	Підпис	Дата				
Розроб.		Крамар Д. О.			Звіт з лабораторної роботи		Лім.	Арк.
Перевір.		Фуріхата Д. В.						1
Керівник							Аркушів	
Н. контр.							ZZ	
Зав. каф.							ФІКТ Гр. ІПЗ-24-3[1]	

3.2. Підключення Mongoose та оновлення server.ts

Встановіть Mongoose та налаштуйте підключення до бази даних перед стартом сервера.

Вимоги:

- Встановіть mongoose.
- Встановіть dotenv та підключіть його на початку server.ts.
- Створіть src/config/database.ts з асинхронною функцією підключення до бази даних.
- Зчитує MONGODB_URI зі змінних середовища.
- Реєструє обробники подій помилки та відключення на об'єкті з'єднання Mongoose.
- Оновіть server.ts: спочатку підключіться до бази даних, і лише потім запускайте сервер. При помилці підключення — завершіть процес з кодом помилки.
- Додайте graceful shutdown: обробіть сигнал SIGTERM і закрийте з'єднання з базою даних перед виходом.

```
// database.ts
import mongoose from 'mongoose';

export async function connectDB(): Promise<void> {
  const uri = process.env.MONGODB_URI;

  if (!uri) {
    console.error('Error: MONGODB_URI not found in environment variables');
    process.exit(1);
  }

  try {
    await mongoose.connect(uri);
    console.log('MongoDB Successfully connected');
  } catch (error) {
    console.error('DB error:', error);
    throw error;
  }
}
```

		Крамар Д. О.			ДУ «Житомирська політехніка».25.121.13.000 – Лр1	Арк.
		Фуріхата Д. В.				2
Змн.	Арк.	№ докум.	Підпис	Дата		

```
}  
}
```

```
mongoose.connection.on('error', (err) => {  
  console.error('MongoDB error:', err);  
});
```

```
mongoose.connection.on('disconnected', () => {  
  console.warn('MongoDB disconnected');  
});
```

```
// server.ts  
import app from "./app";  
import dotenv from 'dotenv';  
import { connectDB } from "./config/database";  
import mongoose from "mongoose";
```

```
dotenv.config();
```

```
const PORT = process.env.PORT || 3000;
```

```
async function startServer() {  
  try {  
    await connectDB();  
  
    const server = app.listen(PORT, () => {  
      console.log(`http://localhost:${PORT}`);  
    });
```

```
    const gracefulShutdown = async (signal: string) => {  
      console.log(`\n...`);
```

```
      await mongoose.connection.close();
```

		Крамар Д. О.			ДУ «Житомирська політехніка».25.121.13.000 – Лр1	Арк.
		Фуріхата Д. В.				3
Змн.	Арк.	№ докум.	Підпис	Дата		

```

    console.log('Closed.');
```

```

    server.close() => {
      console.log('Stoped.');
```

```

      process.exit(0);
    });
  };

  process.on('SIGTERM', () => gracefulShutdown('SIGTERM'));

  process.on('SIGINT', () => gracefulShutdown('SIGINT'));

  } catch (error) {
    console.error('DB error:', error);
    process.exit(1);
  }
}

startServer();
```

3.3. Mongoose-схема та модель

Створіть Mongoose-схему для вашої сутності, яка відображає ті самі атрибути, що й Zod-схема.

Вимоги:

- Створіть `src/models/{entity}.model.ts`.
- Оголосіть TypeScript-інтерфейс для вашої сутності з усіма її полями (без `_id` Mongoose додає його автоматично).
- Опишіть Mongoose-схему на основі цього інтерфейсу з увімкненим автоматичним управлінням датами (`timestamps: true`). Для кожного поля вкажіть тип, обов'язковість, перелік дозволених значень, дефолтне значення та обмеження на довжину/розмір тощо відповідно до Zod-схеми з лаб. 3.
- Zod-схеми залишаються незмінними вони валідують HTTP-запити, Mongoose-схема забезпечує цілісність у БД.

		Крамар Д. О.			ДУ «Житомирська політехніка».25.121.13.000 – Лр1	Арк.
		Фуріхата Д. В.				4
Змн.	Арк.	№ докум.	Підпис	Дата		

- Реалізуйте щонайменше 1 кастомний валідатор для одного з полів функція-предикат з описовим повідомленням про помилку.
- Реалізуйте щонайменше 1 віртуальну властивість computed поле, що не зберігається в БД, але обчислюється на основі наявних полів. Увімкніть включення віртуальних властивостей у JSON-серіалізацію, щоб вони потрапляли до API-відповідей.

```
// category.model.ts
import { Document, Schema, model } from 'mongoose';

export interface ICategory {
  name: string;
  description?: string;
  type: 'income' | 'expense';
  color: string;
  userId: number;
}

export interface ICategoryDocument extends ICategory, Document {
  createdAt: Date;
  updatedAt: Date;
  readonly displayInfo: string;
}

const categorySchema = new Schema<ICategoryDocument>(
  {
    name: {
      type: String,
      required: [true, 'Назва не може бути порожньою'],
      trim: true,
      minlength: [1, 'Назва не може бути порожньою'],
      maxlength: [100, 'Назва занадто довга']
    },
    description: {
      type: String,
```

		Крамар Д. О.			ДУ «Житомирська політехніка».25.121.13.000 – Пр1	Арк.
		Фуріхата Д. В.				5
Змн.	Арк.	№ докум.	Підпис	Дата		

```

        trim: true,
        maxLength: [500, 'Опис занадто довгий'],
        required: false
    },
    type: {
        type: String,
        required: [true, 'Тип є обов\'язковим'],
        enum: {
            values: ['income', 'expense'],
            message: '{VALUE} не є валідним типом категорії'
        }
    },
    color: {
        type: String,
        required: [true, 'Колір є обов\'язковим'],
        match: [/^#[0-9A-Fa-f]{6}$/, 'Невірний формат кольору, очікується HEX (наприклад, #FF0000)']
    },
    userId: {
        type: Number,
        required: [true, 'ID користувача є обов\'язковим'],
        min: [1, 'ID користувача має бути додатним числом'],
        validate: {
            validator: function (value: number) {
                return Number.isInteger(value);
            },
            message: 'ID користувача ({VALUE}) має бути цілим числом'
        }
    },
    {
        timestamps: true,
        toJSON: { virtuals: true },
        toObject: { virtuals: true }
    }
);

```

		Крамар Д. О.			ДУ «Житомирська політехніка».25.121.13.000 – Лр1	Арк.
		Фуріхата Д. В.				6
Змн.	Арк.	№ докум.	Підпис	Дата		

```
categorySchema.virtual('displayInfo').get(function (this: ICategoryDocument) {
  const typeTranslated = this.type === 'income' ? 'Дохід' : 'Витрата';
  return `${this.name} (${typeTranslated}) [Колір: ${this.color}]`;
});
```

```
export const CategoryModel = model<ICategoryDocument>('Category', categorySchema);
```

3.4. Заміна сховища на Mongoose

Замініть Map-сховище з лаб. 3 на виклики Mongoose Model, зберігши той самий API-контракт.

Вимоги:

- Замініть весь вміст storage-модуля (src/storage/{entity}.ts) на функції, що використовують EntityModel.
- MongoDB генерує _id (ObjectId) автоматично ручна генерація ідентифікатора та конструювання об'єкта більше не потрібні.
- Кожна операція реалізується відповідним методом моделі. При оновленні переконайтесь, що метод повертає оновлений документ і запускає валідатори схеми.
- Збережіть (або розширте за необхідності) ті самі статус-коди відповідей: 200, 201, 204, 400, 404.
- Функцію скидання сховища (reset) можна видалити в тестах використовуватиметься очищення через модель.

```
// MongoCategoryRepository.ts
```

```
import { ICategoryRepository, PaginatedResult, PaginationOptions } from './ICategoryRepository';
```

```
import { CategoryEntity, CreateCategoryInput, UpdateCategoryInput } from '../schemas/category.schema';
```

```
import { CategoryModel } from '../models/category.model';
```

		Крамар Д. О.			ДУ «Житомирська політехніка».25.121.13.000 – Лр1	Арк.
		Фуріхата Д. В.				7
Змн.	Арк.	№ докум.	Підпис	Дата		

```
export class MongoCategoryRepository implements ICategoryRepository {
```

```
  async findMany(
```

```
    filters?: { userId?: number; type?: string },
```

```
    options?: PaginationOptions
```

```
  ): Promise<PaginatedResult<CategoryEntity>> {
```

```
    const query: any = {};
```

```
    if (filters?.userId !== undefined) query.userId = filters.userId;
```

```
    if (filters?.type !== undefined) query.type = filters.type;
```

```
    const page = options?.page && options.page > 0 ? options.page : 1;
```

```
    const limit = options?.limit && options.limit > 0 ? options.limit : 10;
```

```
    const skip = (page - 1) * limit;
```

```
    let sortObj: any = { createdAt: -1 };
```

```
    if (options?.sort) {
```

```
      const isDescending = options.sort.startsWith('-');
```

```
      const sortField = isDescending ? options.sort.substring(1) : options.sort;
```

```
      sortObj = { [sortField]: isDescending ? -1 : 1 };
```

```
    }
```

```
    const [categories, total] = await Promise.all([
```

```
      CategoryModel.find(query)
```

```
        .sort(sortObj)
```

```
        .skip(skip)
```

```
        .limit(limit),
```

```
      CategoryModel.countDocuments(query)
```

```
    ]);
```

```
    return {
```

```
      data: categories as unknown as CategoryEntity[],
```

```
      pagination: {
```

```
        page,
```

```
        limit,
```

```
        total,
```

		Крамар Д. О.			ДУ «Житомирська політехніка».25.121.13.000 – Лр1	Арк.
		Фуріхата Д. В.				8
Змн.	Арк.	№ докум.	Підпис	Дата		


```

        totalPages: Math.ceil(total / limit)
      }
    };
  }
}

```

```

async findById(id: string): Promise<CategoryEntity | null> {
  const category = await CategoryModel.findById(id);
  return category as unknown as CategoryEntity | null;
}

```

```

async create(data: CreateCategoryInput): Promise<CategoryEntity> {
  const newCategory = await CategoryModel.create(data);
  return newCategory as unknown as CategoryEntity;
}

```

```

async updateById(id: string, data: UpdateCategoryInput): Promise<CategoryEntity | null> {
  const updatedCategory = await CategoryModel.findByIdAndUpdate(
    id,
    data,
    { new: true, runValidators: true }
  );
  return updatedCategory as unknown as CategoryEntity | null;
}

```

```

async deleteById(id: string): Promise<boolean> {
  const result = await CategoryModel.findByIdAndDelete(id);
  return result !== null;
}
}

```

3.5. Обробка помилок MongoDB

Розширте errorHandler для обробки специфічних помилок Mongoose та MongoDB.

		Крамар Д. О.			ДУ «Житомирська політехніка».25.121.13.000 – Пр1	Арк.
		Фуріхата Д. В.				9
Змн.	Арк.	№ докум.	Підпис	Дата		

Вимоги:

- Розширте src/middleware/errorHandler.ts, щоб правильно обробляти специфічні помилки Mongoose та MongoDB з відповідними статус-кодами. Обробку помилок Zod з лаб. 3 збережіть.
- Переконайтеся, що запит до неіснуючого, але валідного ObjectId повертає 404, а запит з невалідним форматом ID 400.

```
// errorHandler.ts
import { Request, Response, NextFunction } from 'express';
import { ZodError } from 'zod';
import mongoose from 'mongoose';

export const errorHandler = (err: Error, req: Request, res: Response, next:
NextFunction) => {
  if (err instanceof ZodError) {
    res.status(400).json({
      status: 'error',
      message: 'Помилка валідації даних запиту',
      issues: err.issues.map(e => ({
        field: e.path.join('.'),
        message: e.message
      })))
    });
    return;
  }

  if (err instanceof mongoose.Error.CastError && err.path === '_id') {
    res.status(400).json({
      status: 'error',
      message: 'Невалідний формат ідентифікатора (ID)'
    });
    return;
  }

  if (err instanceof mongoose.Error.CastError) {
```

		Крамар Д. О.			ДУ «Житомирська політехніка».25.121.13.000 – Лр1	Арк.
		Фуріхата Д. В.				10
Змн.	Арк.	№ докум.	Підпис	Дата		

```

    res.status(400).json({
      status: 'error',
      message: `Невалідний тип даних для поля: ${err.path}`
    });
    return;
  }

  if (err instanceof mongoose.Error.ValidationError) {
    const issues = Object.values(err.errors).map(e => ({
      field: e.path,
      message: e.message
    }));

    res.status(400).json({
      status: 'error',
      message: 'Помилка збереження даних в БД (Валідація Mongoose)',
      issues
    });
    return;
  }

  if (err.name === 'MongoServerError' && (err as any).code === 11000) {
    res.status(409).json({
      status: 'error',
      message: 'Запис з такими унікальними даними вже існує'
    });
    return;
  }

  console.error('Unhandled error:', err);
  res.status(500).json({
    status: 'error',
    message: 'Внутрішня помилка сервера'
  });
};

```

		Крамар Д. О.			ДУ «Житомирська політехніка».25.121.13.000 – Лр1	Арк.
		Фуріхата Д. В.				11
Змн.	Арк.	№ докум.	Підпис	Дата		

3.6. Фільтрація, сортування та пагінація

Переробіть фільтрацію з лаб. 3 на Mongoose Query API та додайте сортування і пагінацію.

Вимоги:

- Збережіть щонайменше 2 query-параметри фільтрації з лаб. 3, переробивши логіку з in-тету фільтрації на умови Mongoose-запиту — використовуйте оператори порівняння, регулярні вирази або перелік значень залежно від типу поля.
- Додайте сортування через query-параметр sort. Визначте конвенцію для позначення напрямку сортування та реалізуйте її послідовно.
- Реалізуйте пагінацію з параметрами page та limit зі зрозумілими значеннями за замовчуванням. Підрахунок загальної кількості документів та вибірку виконуйте паралельно.
- Формат відповіді GET /api/items змінюється: замість плоского масиву об'єктів із полями data та pagination (поточна сторінка, ліміт, загальна кількість, кількість сторінок).
- Збережіть додатковий доменний маршрут з лаб. 3, адаптувавши його для Mongoose.

```
// src/routes/category.routes.ts
import { Router, Request, Response, NextFunction } from 'express';
import { validate } from '../middleware/validate';
import { createCategorySchema, updateCategorySchema } from '../schemas/category.schema';
import { CategoryService } from '../storage/services/CategoryService';

export const createCategoryRouter = (categoryService: CategoryService) => {
  const router = Router();

  router.get('/', async (req: Request, res: Response, next: NextFunction) => {
    try {
      const { userId, type, page, limit, sort } = req.query;
```

		Крамар Д. О.			ДУ «Житомирська політехніка».25.121.13.000 – Пр1	Арк.
		Фуріхата Д. В.				12
Змн.	Арк.	№ докум.	Підпис	Дата		

```

const filters = {
  userId: userId ? Number(userId) : undefined,
  type: type as string | undefined
};

const options = {
  page: page ? Number(page) : 1,
  limit: limit ? Number(limit) : 10,
  sort: sort as string | undefined
};

const result = await categoryService.getAll(filters, options);
res.status(200).json(result);
} catch (error) {
  next(error);
}
});

router.get('/expenses', async (req: Request, res: Response, next: NextFunction) =>
{
  try {
    const { page, limit, sort } = req.query;

    const options = {
      page: page ? Number(page) : 1,
      limit: limit ? Number(limit) : 10,
      sort: sort as string | undefined
    };

    const result = await categoryService.getAll({ type: 'expense' }, options);
    res.status(200).json(result);
  } catch (error) {
    next(error);
  }
});

```

```

router.get('/:id', async (req: Request<{ id: string }>, res: Response, next:
NextFunction) => {
    try {
        const category = await categoryService.getById(req.params.id);
        if (!category) {
            res.status(404).json({ error: 'Категорію не знайдено' });
            return;
        }
        res.status(200).json(category);
    } catch (error) { next(error); }
});

```

```

router.post('/', validate(createCategorySchema), async (req: Request, res: Re-
sponse, next: NextFunction) => {
    try {
        const newCategory = await categoryService.create(req.body);
        res.status(201).json(newCategory);
    } catch (error) { next(error); }
});

```

```

router.patch('/:id', validate(updateCategorySchema), async (req: Request<{ id:
string }>, res: Response, next: NextFunction) => {
    try {
        const updatedCategory = await categoryService.update(req.params.id,
req.body);
        if (!updatedCategory) {
            res.status(404).json({ error: 'Категорію не знайдено' });
            return;
        }
        res.status(200).json(updatedCategory);
    } catch (error) { next(error); }
});

```

```

router.delete('/:id', async (req: Request<{ id: string }>, res: Response, next:
NextFunction) => {
    try {

```

		Крамар Д. О.			ДУ «Житомирська політехніка».25.121.13.000 – Пр1	Арк.
		Фуріхата Д. В.				14
Змн.	Арк.	№ докум.	Підпис	Дата		

```

    const isDeleted = await categoryService.delete(req.params.id);
    if (!isDeleted) {
      res.status(404).json({ error: 'Категорію не знайдено' });
      return;
    }
    res.status(204).send();
  } catch (error) { next(error); }
});

return router;
};

```

3.7. Тестування з mongodb-memory-server

Перепишіть тести для роботи з реальною MongoDB у пам'яті замість in-memory Map.

Вимоги:

- Встановіть mongodb-memory-server як dev-залежність.
- Створіть tests/setup.ts з допоміжними функціями запуску і зупинки тестової БД.
- У кожному тестовому файлі використовуйте хуки beforeAll/afterAll для підключення та afterEach для очищення колекції між тестами.
- Напишіть unit-тести моделі (в окремому блоці describe) — перевірте дефолти, автоматичні дати, правила валідації, віртуальні властивості тощо.
- Перепишіть інтеграційні тести з лаб. 3 для роботи з MongoDB покрийте всі маршрути, включаючи граничні випадки специфічні для MongoDB.
- Фінальна кількість тестів — не менша за лаб. 3.
- Спробуйте досягти 100% покриття тестами коду, що стосується логіки проєкту.

```

// setup.ts
import mongoose from 'mongoose';
import { MongoMemoryServer } from 'mongodb-memory-server';

```

		Крамар Д. О.			ДУ «Житомирська політехніка».25.121.13.000 – Лр1	Арк.
		Фуріхата Д. В.				15
Змн.	Арк.	№ докум.	Підпис	Дата		

```
let mongoServer: MongoMemoryServer;
```

```
export const connectTestDB = async () => {  
  mongoServer = await MongoMemoryServer.create();  
  const uri = mongoServer.getUri();  
  await mongoose.connect(uri);  
};
```

```
export const closeTestDB = async () => {  
  await mongoose.connection.dropDatabase();  
  await mongoose.connection.close();  
  await mongoServer.stop();  
};
```

```
export const clearTestDB = async () => {  
  const collections = mongoose.connection.collections;  
  for (const key in collections) {  
    const collection = collections[key];  
    await collection.deleteMany({});  
  }  
};
```

		Крамар Д. О.			ДУ «Житомирська політехніка».25.121.13.000 – Лр1	Арк.
		Фуріхата Д. В.				16
Змн.	Арк.	№ докум.	Підпис	Дата		